

Learn ESP32 for Home Automation

Lesson 5: HTTP Requests and Simple Web Server

HTTP Requests and Simple Web Server - Study Notes

Key Concepts

1. **HTTP GET request** – the most common request type used by browsers to retrieve resources (e.g., HTML pages, images, or API data).
2. **URL query parameters** – data appended to a URL after a ``?`` (e.g., ``?led=on``) that the server can read to decide what action to take.
3. **Embedded web server on a microcontroller** – lightweight server (often using libraries like `ESP8266WebServer`, `WiFiServer`, or `Arduino /Ethernet`) that listens for incoming connections and serves HTML or handles control commands.
4. **GPIO control from web requests** – mapping a request (e.g., ``GET /led?state=on``) to a physical action such as turning an LED on or off.
5. **Basic HTML/JavaScript UI for toggling** – a simple page with buttons or links that send GET requests to the server without needing a full stack framework.

Summary

In this lesson you learned how to turn a microcontroller (e.g., an ESP8266, ESP32, or Arduino with Ethernet) into a tiny web server that can receive HTTP GET requests and use them to control an LED. The server listens on a specific port (usually 80), parses the request URL, extracts query parameters, and sets the appropriate GPIO pin high or low.

A minimal HTML page is served to the client, containing two buttons—**ON** and **OFF**—that each issue a GET request with a query string (``/led?state=on`` or ``/led?state=off``). When the microcontroller receives the request, it toggles the LED accordingly and optionally returns a short status page confirming the new state. This pattern introduces the fundamentals of web based IoT control: handling HTTP, serving static content, and interfacing network commands with hardware.

Important Points

- **HTTP GET vs. POST** – For simple toggling, GET is sufficient because the request does not modify server side data beyond the LED state; POST is used when sending larger or sensitive payloads.
- **Parsing the request** – Most Arduino style libraries give you the request line (``GET /path?param=value HTTP/1.1``). Use ``String`` methods (``indexOf``, ``substring``) or the library's built in argument parser to extract ``state``.
- **Safety** – Never expose the microcontroller to the open internet without authentication; for learning, keep it on a local Wi Fi network.
- **HTML button implementation** – You can use ``<a href="/led?state=on">ON</a>`` or a ``<button>`` with JavaScript ``fetch('/led?state=off')``. The former reloads the page; the latter can update the UI without a full refresh.

- ****Debounce the LED**** – If the request is sent repeatedly (e.g., rapid button clicks), add a short software debounce or ignore duplicate state changes.
- ****Response codes**** – Return `200 OK` for successful toggles; you may also send `404 Not Found` for unknown paths to keep the server tidy.
- ****Memory considerations**** – Keep the HTML string short; large pages can exhaust RAM on low memory boards.

Examples

1. Arduino style code (ESP8266) – handling a GET request to toggle an LED

```
```cpp
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
const char* ssid = "yourSSID";
const char* password = "yourPASS";
const int LED_PIN = D4; // built in LED on many ESP boards
ESP8266WebServer server(80); // HTTP server on port 80
// Serve the control page
void handleRoot() {
 String html = "<!DOCTYPE html><html><head><title>LED Control</title></head>"
 "<body><h1>LED Toggle</h1>"
 "<button>ON</button>"
 "<button>OFF</button>"
 "</body></html>";
 server.send(200, "text/html", html);
}
// Parse the query and set the LED
void handleLED() {
 if (server.hasArg("state")) {
 String state = server.arg("state");
 if (state == "on") digitalWrite(LED_PIN, LOW); // LOW = LED on for built in LED
 if (state == "off") digitalWrite(LED_PIN, HIGH);
 server.send(200, "text/plain", "LED is now " + state);
 } else {
 server.send(400, "text/plain", "Bad Request: missing 'state'");
 }
}
void setup() {
 pinMode(LED_PIN, OUTPUT);
 digitalWrite(LED_PIN, HIGH); // start LED off
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) delay(500);
}
```

```

server.on("/", handleRoot);
server.on("/led", handleLED);
server.begin();
Serial.begin(115200);
Serial.println("Server started, IP: " + WiFi.localIP().toString());
}
void loop() {
 server.handleClient(); // process incoming requests
}
...

```

**\*Explanation\*:**

- The device connects to Wi Fi, starts a server on port /80, and registers two routes (`/` and `/led`).
- `handleRoot()` sends a tiny HTML page with two buttons. Clicking a button triggers a GET request to `/led?state=on` or `/led?state=off`.
- `handleLED()` reads the `state` query parameter, writes the appropriate level to the LED pin, and replies with a plain text confirmation.

---

## 2. Minimal HTML + JavaScript (no page reload)

```

<<<html
<!DOCTYPE html>
<html>
<head>
 <title>LED Toggle (AJAX)</title>
 <script>
 function toggle(state) {
 fetch('/led?state=' + state)
 .then(r => r.text())
 .then(txt => document.getElementById('status').innerText = txt)
 .catch(err => console.error(err));
 }
 </script>
</head>
<body>
 <h1>LED Control</h1>
 <button onclick="toggle('on')">ON</button>
 <button onclick="toggle('off')">OFF</button>
 <p id="status">Current status unknown</p>
</body>
</html>
...

```

**\*Explanation\*:**

- The page uses the `fetch` API to send a GET request without reloading.

- The server's `/led` handler can stay exactly the same; it returns a short text that the script places into the `<p id="status">` element, giving instant feedback.

---

## Quick Review

1. **What part of an HTTP GET request contains the data you need to decide whether to turn the LED on or off?**
2. **Why is it safe to use a GET request for this simple LED toggle, but not for sending passwords?**
3. **Name two ways to provide user feedback after the LED state changes.**
4. **What does `server.handleClient()` do in the Arduino loop?**
5. **If you wanted to protect the endpoint from the internet, what basic step could you add?**

---

## Further Reading

- **HTTP methods** – deeper dive into GET, POST, PUT, DELETE and when to use each.
- **Security for IoT web servers** – basic authentication, HTTPS with self signed certificates, and firewalling.
- **WebSockets** – real time bidirectional communication for status updates without polling.
- **ESPAsyncWebServer library** – non blocking server handling for higher performance projects.
- **RESTful API design** – structuring endpoints (`/led/on`, `/led/off`) and using proper status codes.

---

\*End of study notes.\*

